

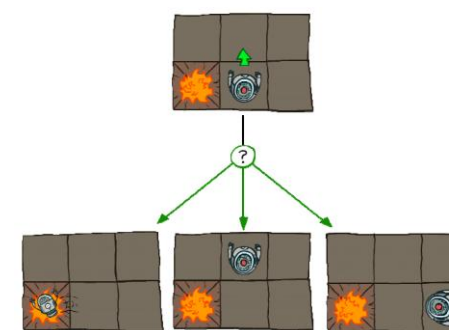
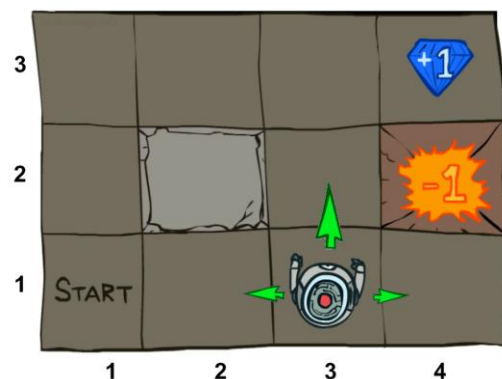
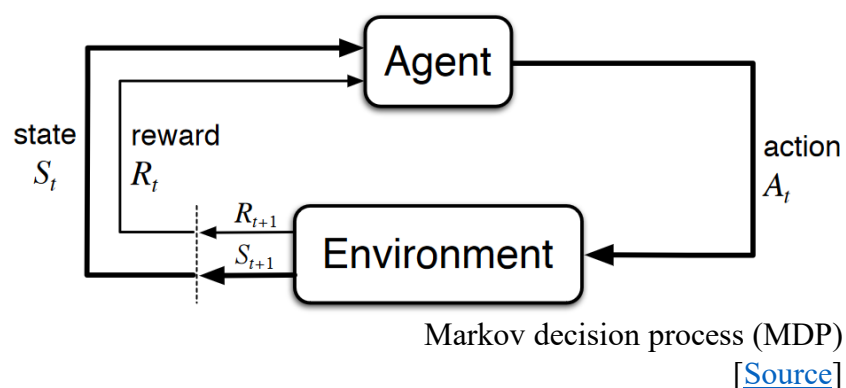
Guest Lecture

Reward Shaping for Reinforcement Learning and Its Applications

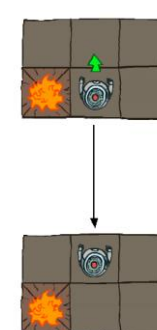
Ma Haozhe

Markov Decision Process and Reinforcement Learning

1. Reinforcement Learning (RL) solves **sequential decision-making** problems.
 1. The learning paradigm is like how human learns from our real world.
 2. An agent **refines its behaviors by interacting with an environment**.
2. The **interaction** between agent and environment is modeled as a **Markov Decision Process (MDP)**



Transition probability



deterministic

- S : state space
- A : action space
- T : transition function: $P(s'|s, a)$
- R : reward function
- γ : discount factor

- State space: the location of the robot. $S = \{(1,1), (1,2), \dots, (4,3)\}$
- Action space: moving to four directions. $A = \{\leftarrow, \rightarrow, \uparrow, \downarrow\}$
- Transition functions: $P(s'|s, a)$
- Reward functions: $R((4,3)) = 1, R((4,2)) = -1, R(\dots) = 0$

Review



- **Policy:**

$$\pi: S \rightarrow A, \pi(a|s) = \Pr(A_t = a \mid S_t = s)$$

- **Value** of a specific policy π for state s , the expected cumulative discounted rewards starting from s :

$$V^\pi(s_\tau) = \mathbb{E}[\underbrace{R(s_\tau) + \gamma R(s_{\tau+1}) + \gamma^2 R(s_{\tau+2}) + \cdots}_{\text{Return}}] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R(s_{\tau+k}) \right]$$

- **Q-value** defines the value of each state-action pair, starting from s , forcing to do action a .

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$$

- Reinforcement learning: to learn an **optimal policy** that maximize the Q-values for each state.

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

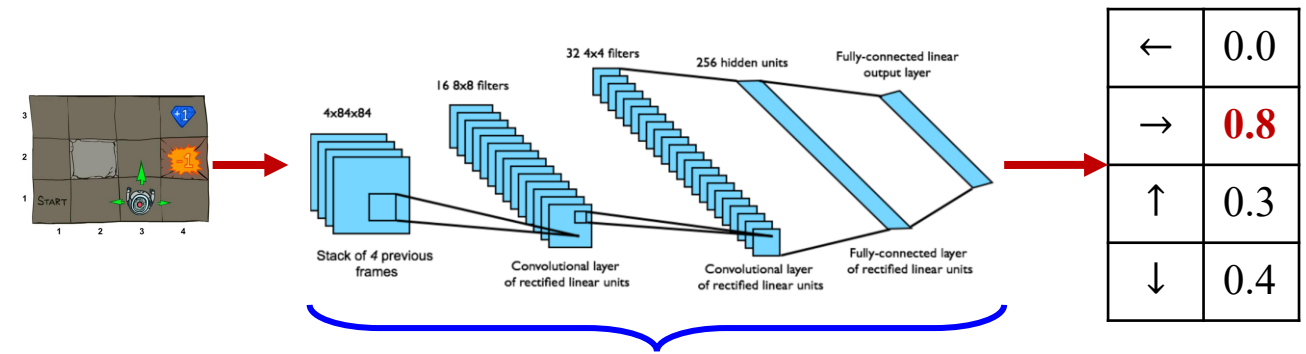
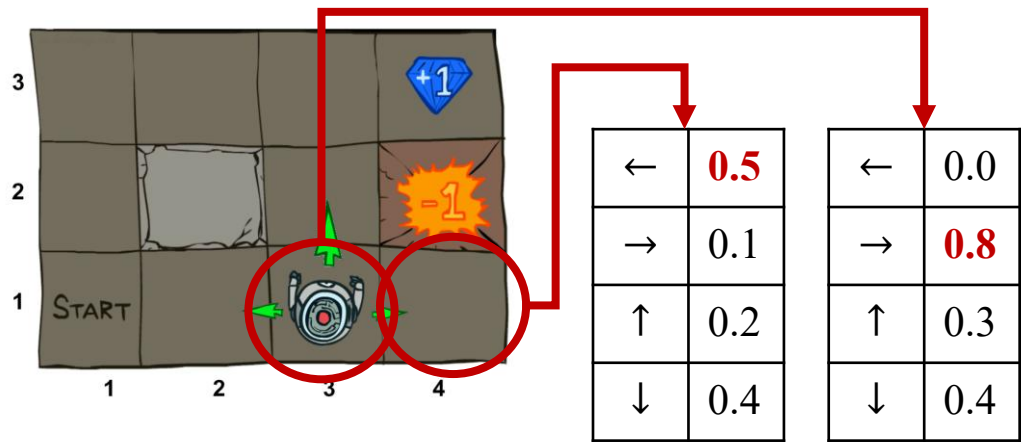
Review

- Q-Learning

$$\underbrace{\pi^*(s)}_{\text{To learn the optimal policy}} = \arg \max_a \underbrace{Q^*(s, a)}_{\text{To learn the optimal (exact) Q-function}}$$

To learn the optimal policy To learn the optimal (exact) Q-function

- Represent the Q-function: $\hat{Q}_\theta(s, a)$
 - If represent the $\hat{Q}_\theta(s, a)$ as a table: tabular Q-learning
 - If represent the $\hat{Q}_\theta(s, a)$ as a linear/nonlinear function: Function approximation Q-learning
 - If represent the $\hat{Q}_\theta(s, a)$ as a deep neural network: DQN



Let network estimate which state and the corresponding Q-values

Review

- Learn Q-function $\hat{Q}_\theta(s, a) \rightarrow \underbrace{Q^*(s, a)}$

Learning target, i.e., y_t

- Monte Carlo: $y_t^{MC} = G_t = \underbrace{r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots}$

Collect all rewards of a trajectory

- Temporal Difference (TD):

$$y_t^{MC} = G_t = r_t + \underbrace{\gamma r_{t+1} + \gamma^2 r_{t+2} + \dots}$$

Estimate by the currently/previously learned $\hat{Q}_{\theta_{old}}(s, a)$

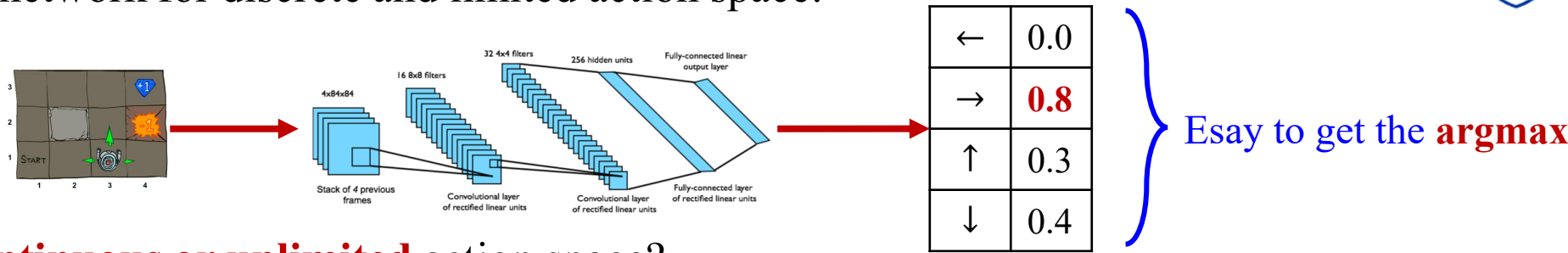


$$y_t^{\text{SARSA}} = G_t = r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1})$$

$$y_t^{\text{Q-Learning}} = G_t = r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a')$$

Review

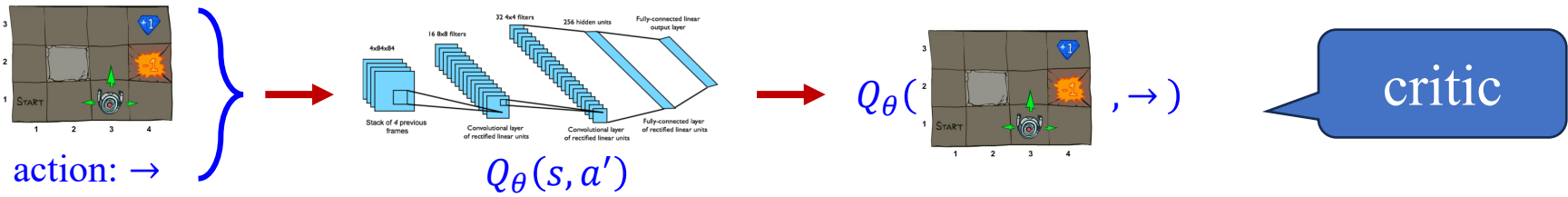
- Q-function/Q-network for discrete and limited action space:



- How about **continuous or unlimited** action space?

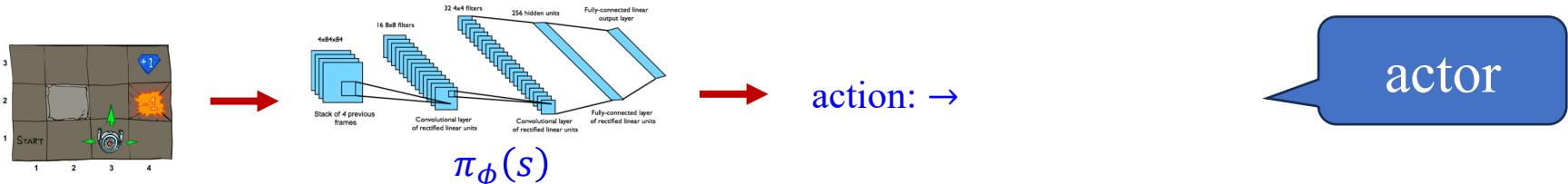
- Network/table **can't list all actions**:

⇒ Input **both state and action** into network, only output the Q-value



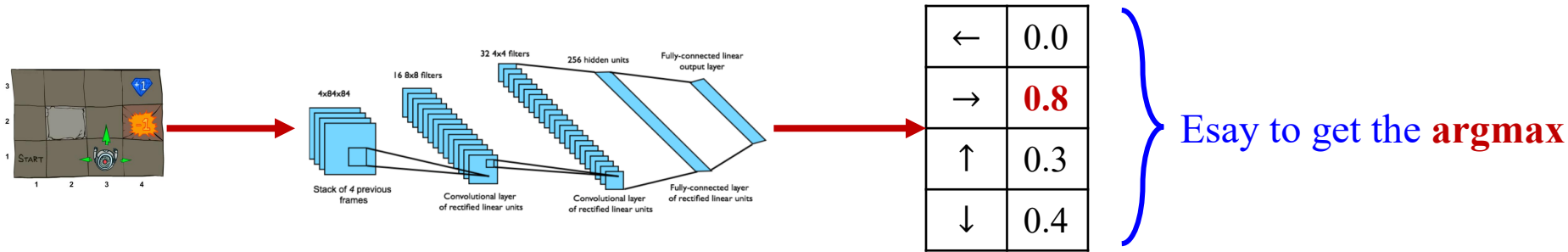
- Can't **compute the argmax**

⇒ Learn **another function to estimate argmax** : $\pi_{\phi}(s) \approx \arg \max_{a'} Q_{\theta}(s, a')$

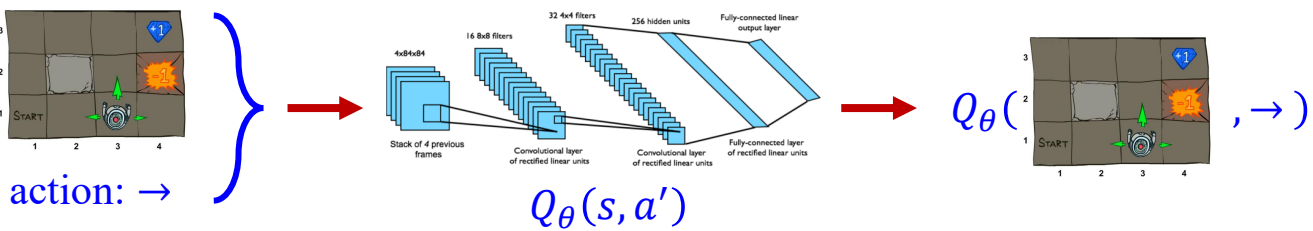


Reward Matters

- Q-learning => Q-function



- Actor-critic => Value/Q-value function



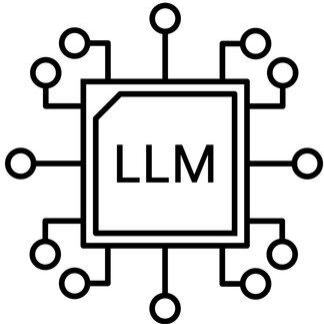
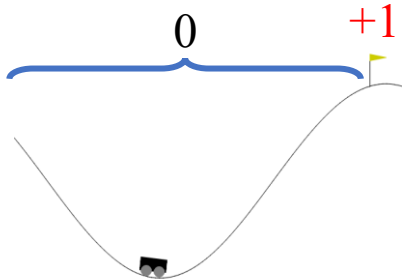
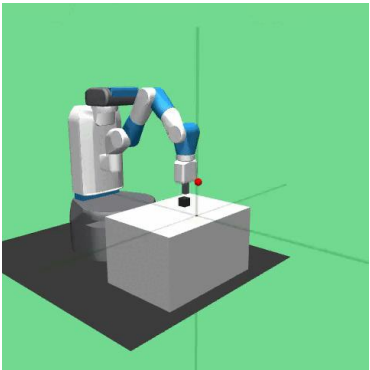
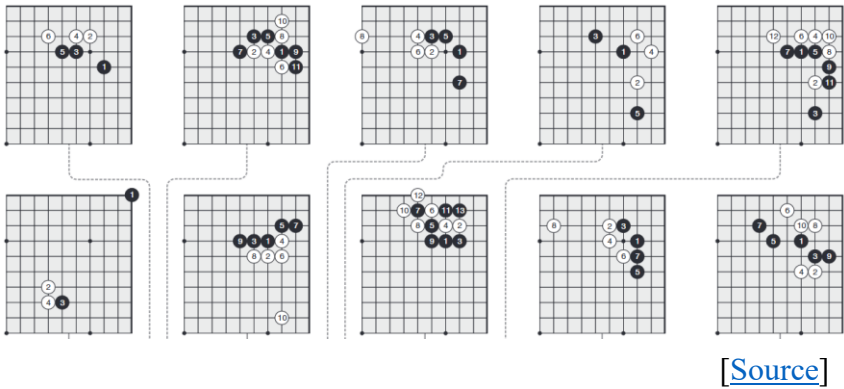
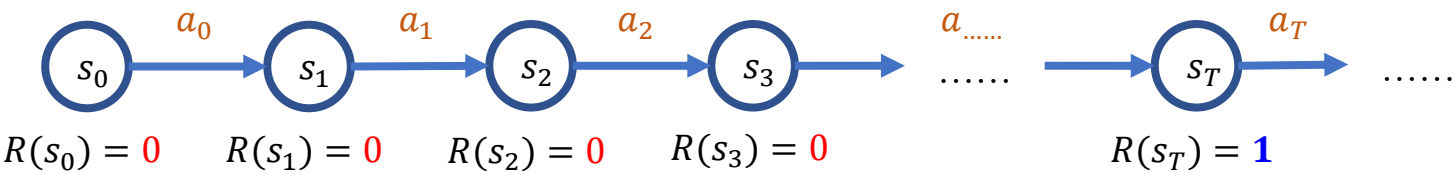
Return: $G_t = R(s_t) + \gamma R(s_{t+1}) + \gamma^2 R(s_{t+2}) + \dots$

Reward comes from interaction with environment

Challenge: Native Reward Models

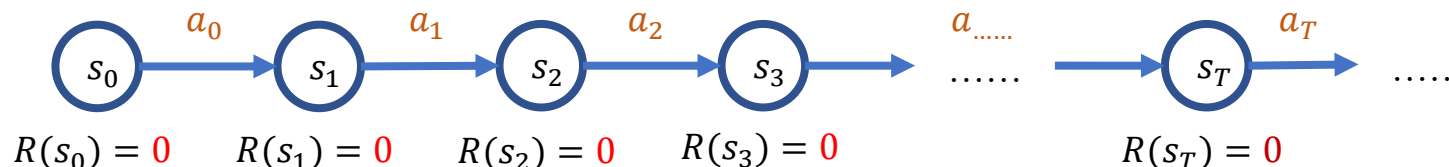
Native environmental rewards are typically:

- sparse, delayed, and non-informative;
- only indicating the completion of the overall task (at most some milestones).
- 0 rewards for all in-process states, providing no information.



What will happen if we learn policies in a sparse-reward environment?

■ start ■ goal, the only reward in the whole MDP ■ wall



$$y_t^{MC} = G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = 0$$

$$\hat{Q}_\theta(s_t, a) \rightarrow \underbrace{y_t^{MC}}$$

Target in this sample will be 0

$$y_t^{\text{Q-Learning}} = G_t = \underbrace{r_t}_{\text{red}} + \gamma \max_{a'} Q(s_{t+1}, a') = \text{red} + \gamma \max_{a'} Q(s_{t+1}, a')$$

one-step reward is 0

If $Q \equiv 0$ initialized, then $Q \leftarrow Q$

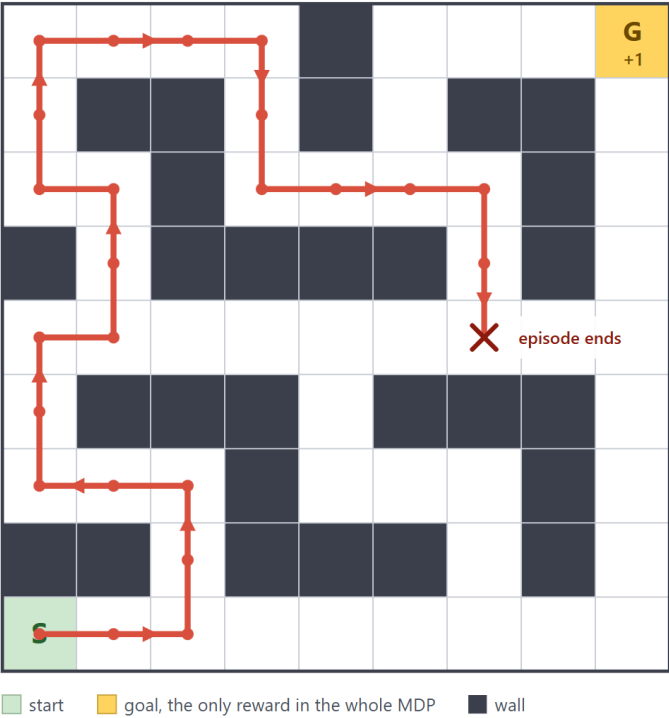
Note:

The **algorithm finally works**, we're focusing on the **sample efficiency**

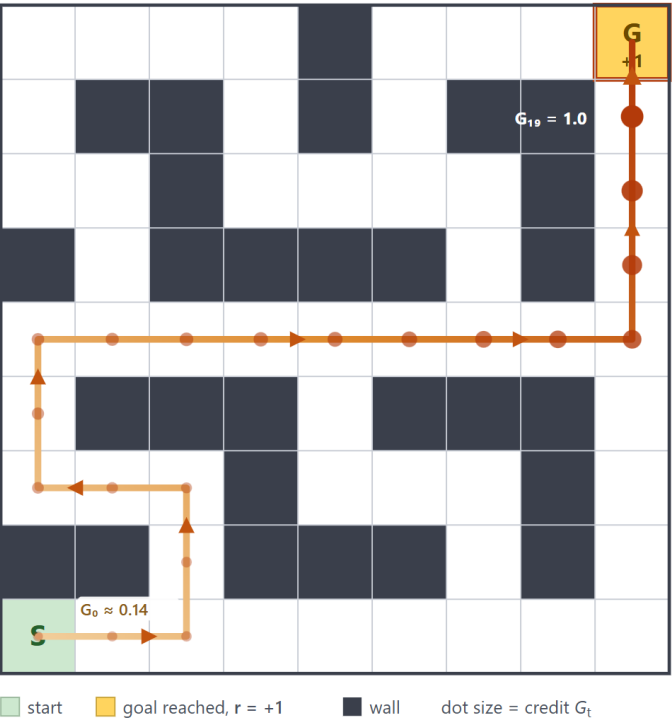
Challenge: Native Reward Models

You'll notice: the RL algorithms only “**begin to learn**” after sampling a successful reward.

Sparse reward: one sampled episode, zero feedback



Sparse reward: the rare successful episode



$$y_t^{MC} = 0 + \gamma \times 0 + \dots + \gamma^{20} \times 1 \approx 0.12$$



$$\hat{Q}_\theta(s_t, a) \rightarrow 0.12$$

Start to learn!

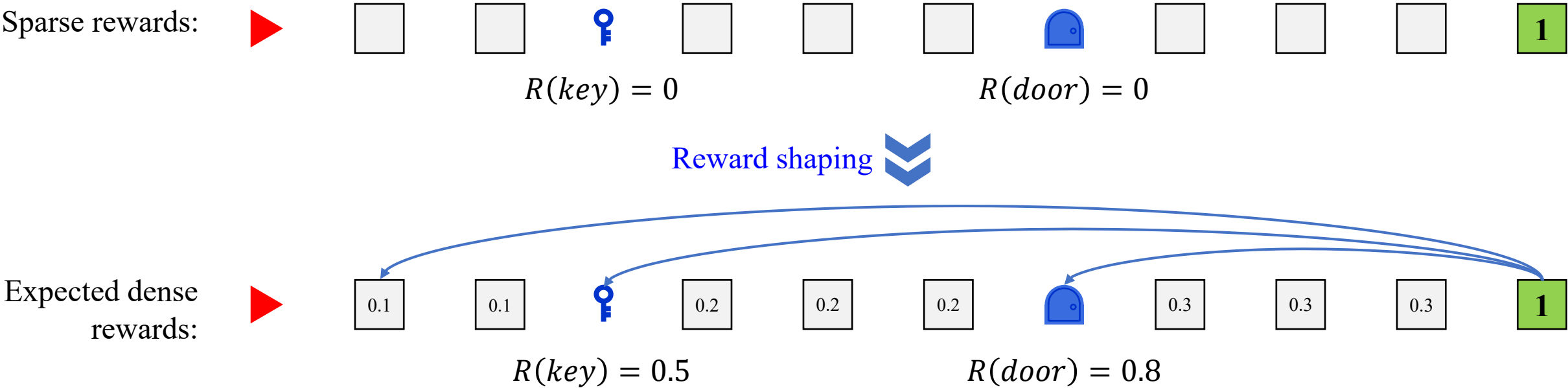
Note:
The **algorithm finally works**, we're focusing on the **sample efficiency**

Reward Shaping: A Promising Solution

- **Native reward models:** sparse, delayed, non-informative, ...
- **Reward Shaping (RS):** Reshape/rebuild the native reward model to more informative reward structures.

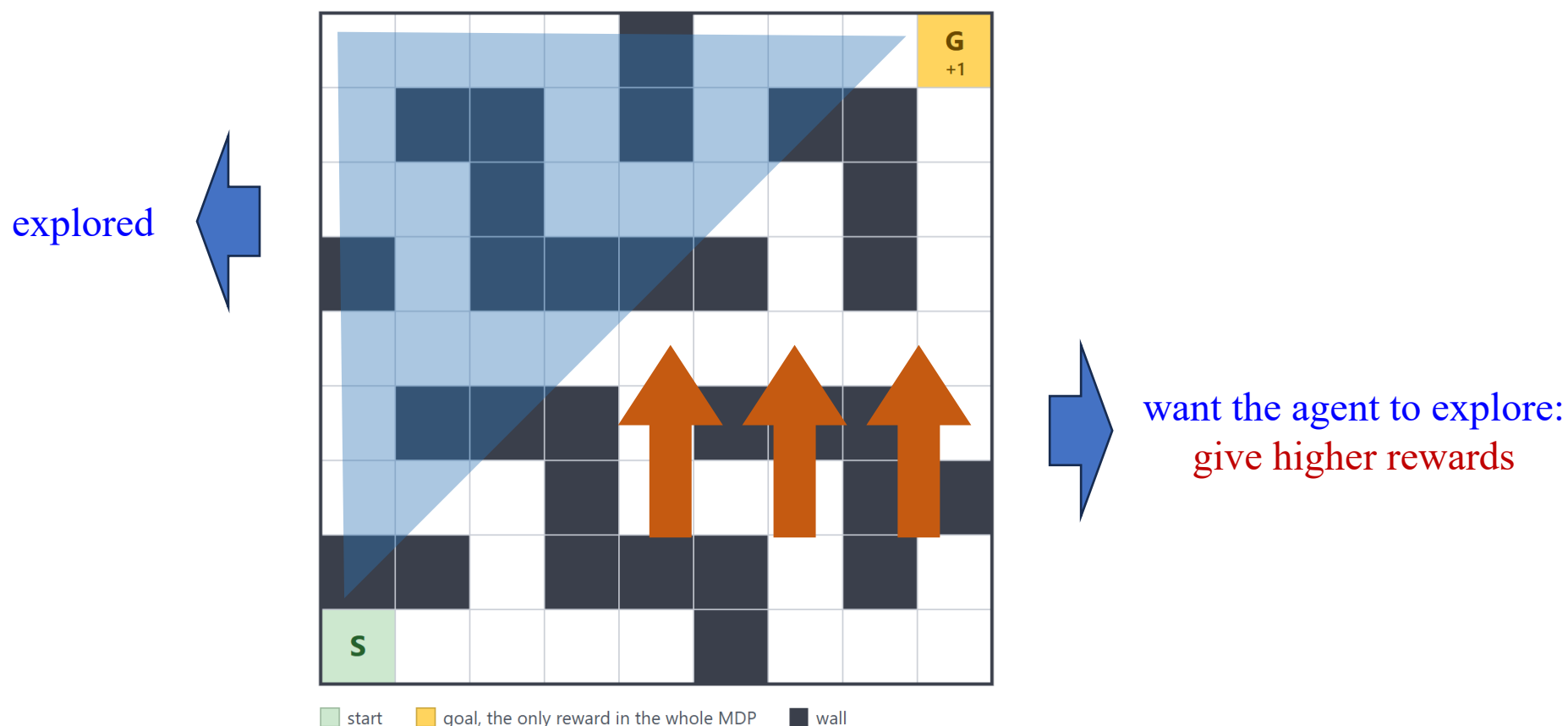
$$R^{new}(\cdot) = \alpha R^{env}(\cdot) + \beta R^{sha}(\cdot)$$

- **Reshaping rewards** allows us to explicitly **control over the agent's behavior**
- **Main problem:** **how to define/learn/maintain a shaped reward $R^{sha}(\cdot)$**



Reward Shaping for Exploration

- By reshaping rewards, we can control agents to **explore**.
- **Exploration bonus**: Assign higher rewards to under-explored regions.



Reward Shaping for Exploration

- How to reward under-explored states?
 - **Novelty**: assign higher rewards to novel states.

➤ Method #1: For tabular states: count **visiting numbers**:

$$R^{count}(\cdot) = \alpha R^{env}(\cdot) + \frac{\beta}{N(s) + 1}$$

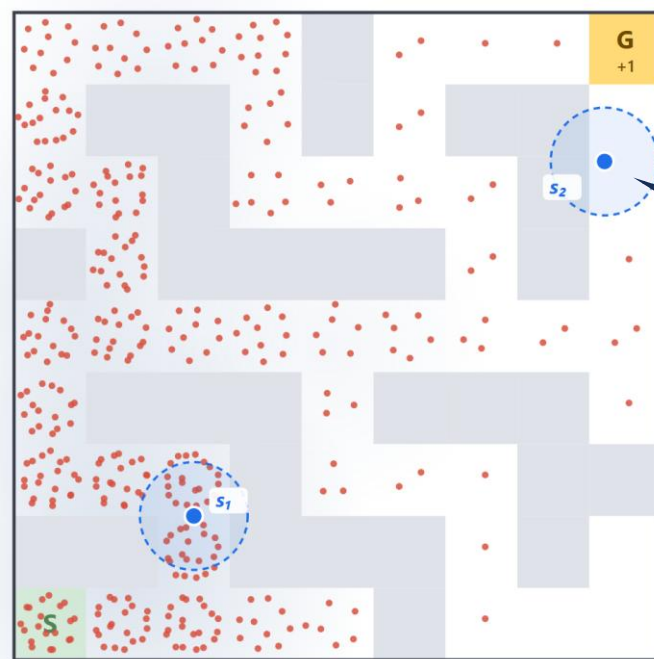
Not divided by 0



Reward Shaping for Exploration

- How to reward under-explored states?
 - Novelty**: assign higher rewards to novel states.
 - Method #2: For continuous states: no state is ever visited twice: **pseudo-count by density model**

$$R^{\text{pseudo-count}}(\cdot) = \alpha R^{\text{env}}(\cdot) + \frac{\beta}{\underbrace{\hat{N}(s) + 1}_{\text{Estimated count}}}$$



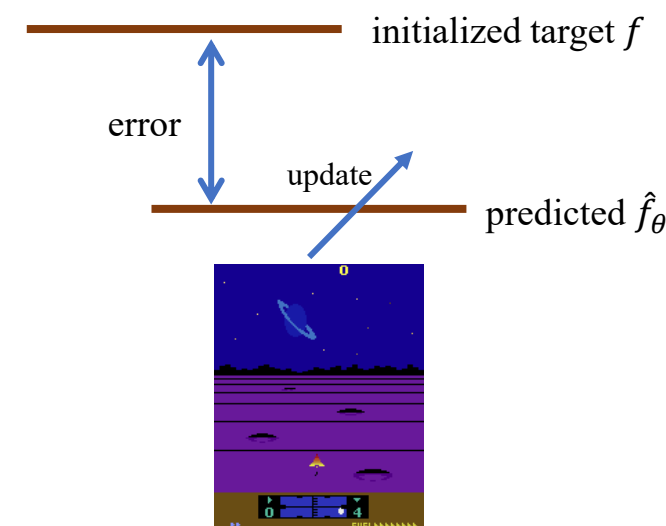
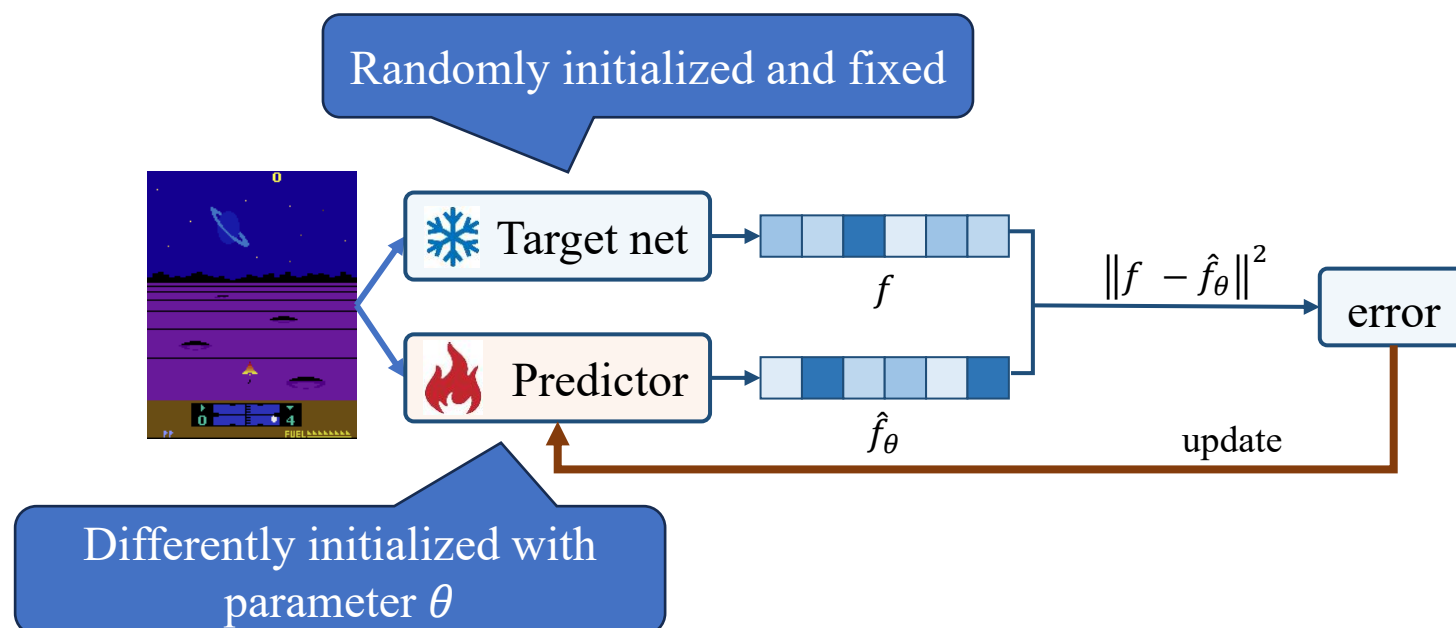
Estimate: how many history visiting counts near the specific state.

Reward Shaping for Exploration

- How to reward under-explored states?
 - Novelty**: assign higher rewards to novel states.
 - Method #3: **Random network distillation** (RND): for high-dimension states: density is expensive to compute.

$$R^{\text{RND}}(\cdot) = \alpha R^{\text{env}}(\cdot) + \beta \underbrace{\|f(s) - \hat{f}_\theta(s)\|^2}_{\text{The error between the predicted value and a randomly fixed value}}$$

The error between the predicted value and a randomly fixed value



[1] Burda Y, Edwards H, Storkey A, et al. Exploration by random network distillation[J]. arXiv preprint arXiv:1810.12894, 2018.

Reward Shaping for Exploration

- **Challenge:** novelty does not align with the original goal.
 - **Noisy-TV problem:** if we put a TV broadcasting random channels in a maze, agent will be attracted by the noisy TV, as **every channel is novel**.

with a noisy TV

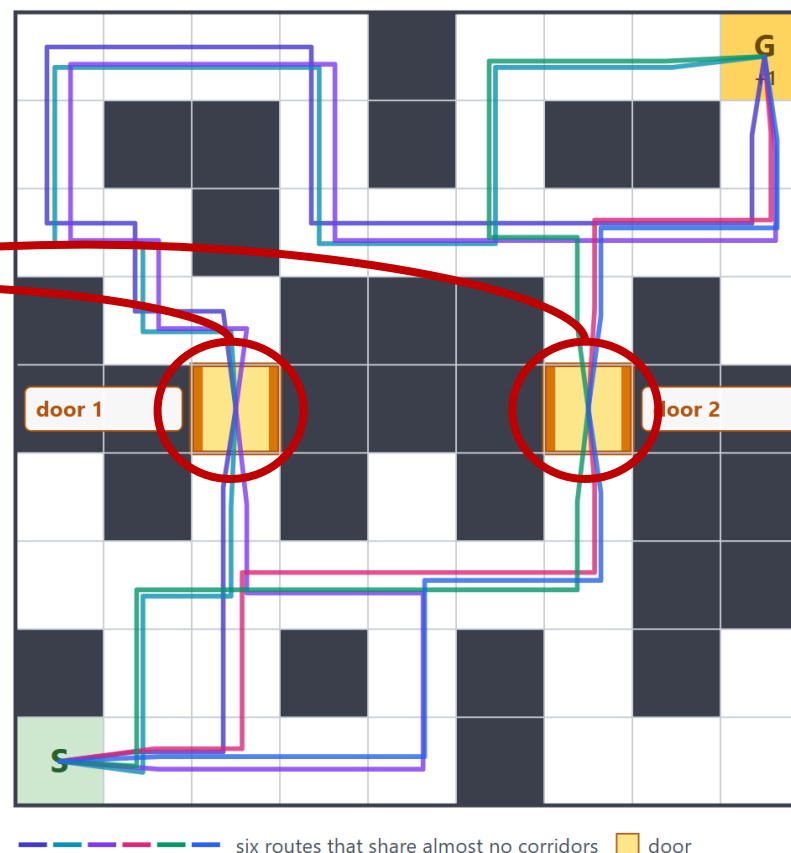
without a noisy TV



Reward Shaping for Exploitation

- By reshaping rewards, we can control agents to **exploit (optimization)**.
- Assign higher rewards to more meaningful/valuable states.

The two doors are “more important”:
give higher rewards

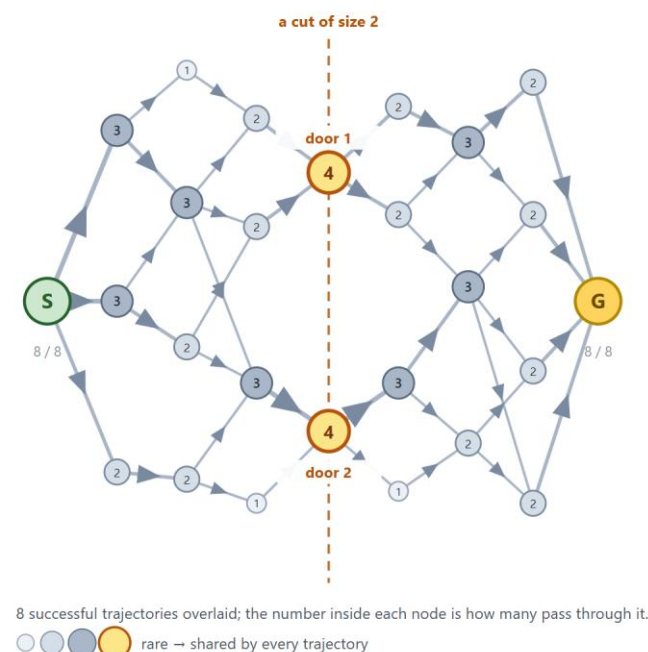


Reward Shaping for Exploitation

- By reshaping rewards, we can control agents to **exploit (optimization)**.
- Assign higher rewards to more meaningful/valuable states.
 - Method #1: Graph/Clustering based: find the **most common** states.



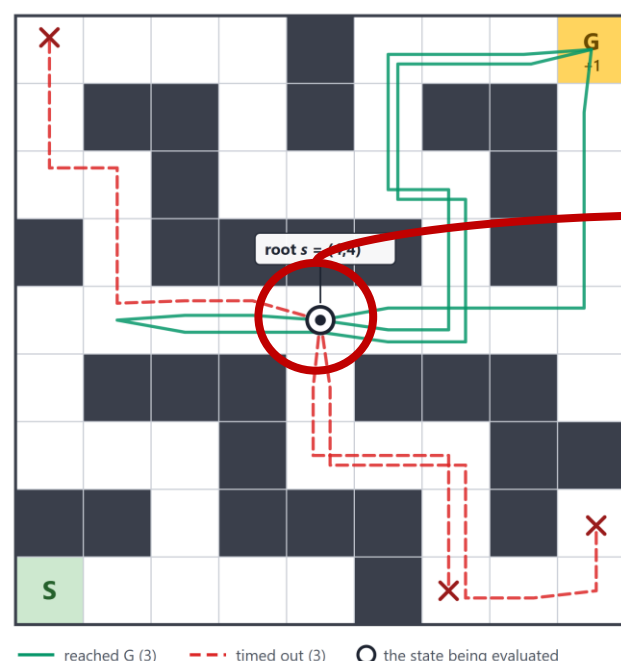
building transition graph
or clustering history states



- Challenge: the **states near the starting point** are always most commonly visited.

Reward Shaping for Exploitation

- By reshaping rewards, we can control agents to **exploit (optimization)**.
- Assign higher rewards to more meaningful/valuable states.
 - Method #2: Tree-like search to estimate the **potential success rate**.

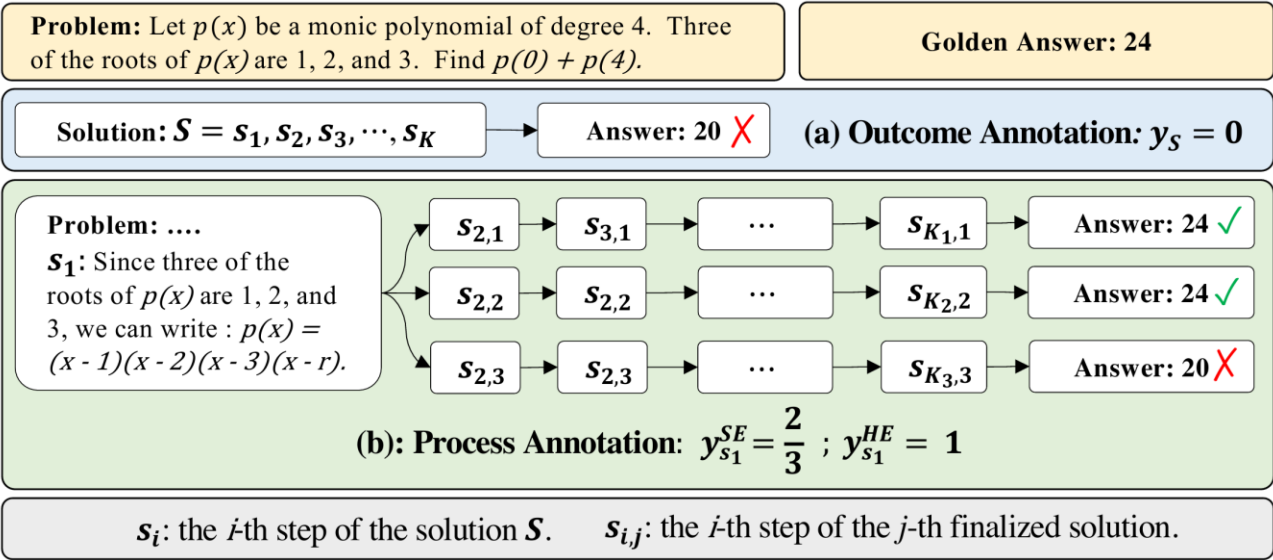


Can fix a specific state (prefix) to
start to rollout

- Challenge: searching is very expensive.

Reward Shaping for Exploitation

- By reshaping rewards, we can control agents to **exploit (optimization)**.
- Assign higher rewards to more meaningful/valuable states.
 - Method #2: Tree-like search to estimate the potential success rate.
 - Example: **Math-Shepherd** reward model in LLM domain.



[1] Wang P, Li L, Shao Z, et al. Math-shepherd: Verify and reinforce llms step-by-step without human annotations[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 9426-9439.

Challenge: Consistent Optimal Policy

- Reward shaping changed the reward function, will the learned optimal policy is consistent with the original one?
- Writing the shaped reward function as potential-difference functions**
- Theoretically guarantee that the optimal policy remains unchanged.

$$R^{new}(\cdot) = \alpha R^{env}(\cdot) + \beta \mathbf{R}^{sha}(\cdot)$$

$$\mathbf{R}^{sha}(s) = \phi(s) - \gamma\phi(s')$$

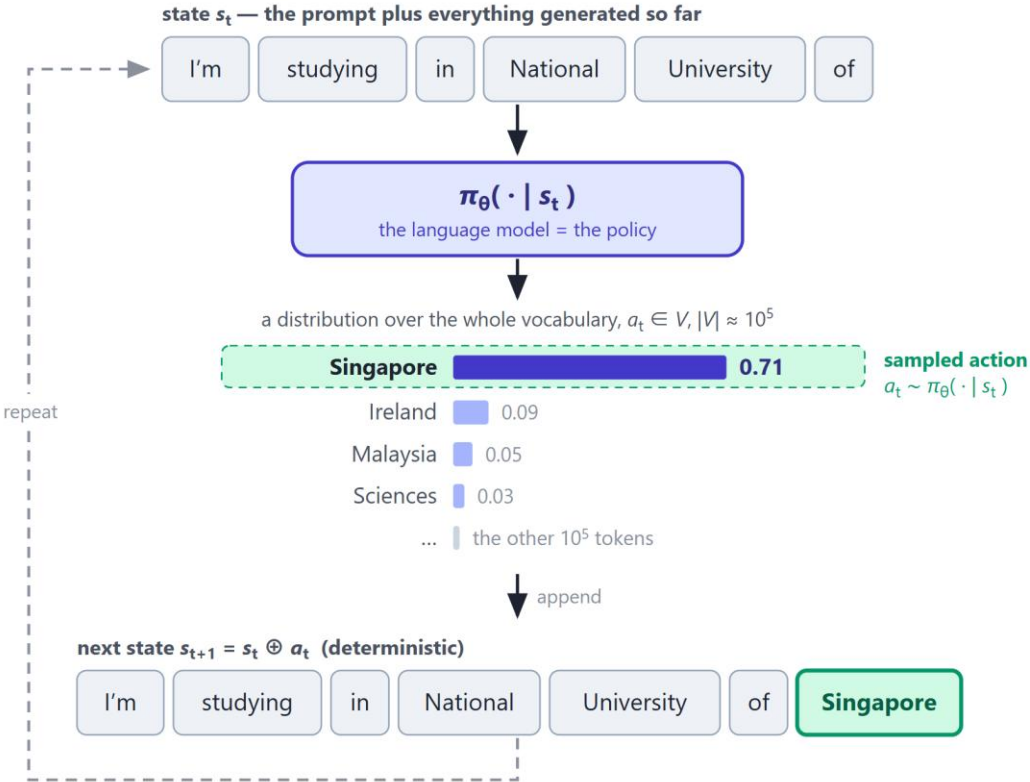
$$G_t = R^{sha}(s) + \gamma R^{sha}(s') + \dots = \phi(s) \underbrace{- \gamma\phi(s') + \gamma(\phi(s') - \gamma\phi(s''))}_{\text{Eliminated}} = \phi(s) - \gamma^2\phi(s'')$$

$$G_t = R^{sha}(s) + \gamma R^{sha}(s') + \gamma^2 R^{sha}(s'') + \dots = \phi(s) \underbrace{- \gamma^2\phi(s'') + \gamma^2(\phi(s'') - \gamma\phi(s'''))}_{\text{Eliminated}} = \phi(s) - \gamma^3\phi(s''')$$

- All other RS methods, including all our methods, **DO NOT offer** theoretical guarantee.
- Most reward-shaping methods are evaluated **empirically** for higher sample efficiency.

Reward Shaping for Open-Ended Environments in LLM

- Reinforcement Learning is widely used in Large Language Models (LLMs)



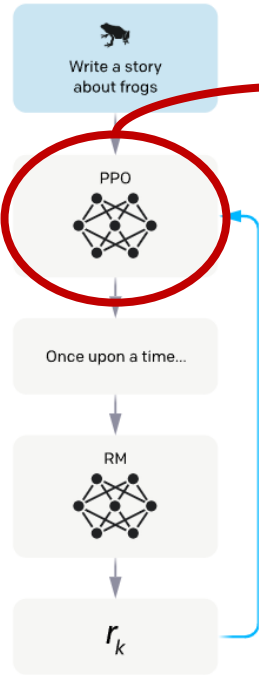
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Reinforcement Learning

Reward Shaping for Open-Ended Environments in LLM



- Reinforcement Learning is widely used in Large Language Models (LLMs)
- Many **open-ended questions** typically **do not have a standard answer** or an environmental reward.
 - Write a short self-introduction for my first day at a new job.
 - Suggest five names for a coffee shop next to a university campus.
 - Draft a LinkedIn headline for a data scientist moving into product.
 - ...
- **Challenge: How to define rewards?**
 - ⇒ **Reinforcement learning from human feedback (RLHF)**

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray et al., “Training language models to follow instructions with human feedback,” Advances in Neural Information Processing Systems, vol. 35, pp. 27 730–27 744, 2022.
- [2] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” Advances in neural information processing systems, vol. 30, 2017.

Reward Shaping for Open-Ended Environments in LLM

- Building a **reward model** in RLHF

- One prompt $\rightarrow$ SFT model $\rightarrow N$ responses.
- Human experts: rank the N response based on preference:

$$res_1 > res_2 > \dots > res_N$$

- Prepare a **reward model**:

- Usually another LLM with the same scale of the SFT model.
- Add a regression head to the final layer, making it a mapping function:

$$r_{\theta}(x, y) \rightarrow \mathbb{R}$$

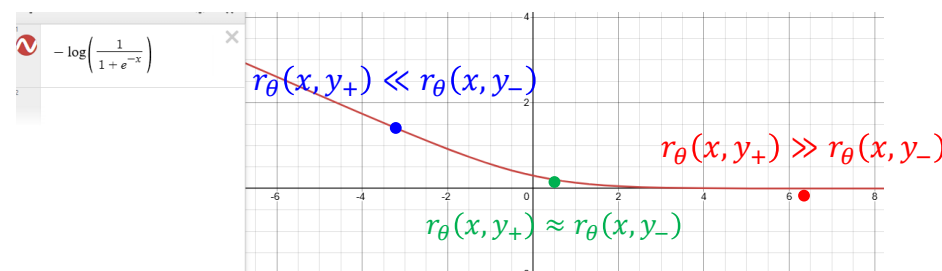
where x is the prompt, y is the response.

- Train the reward model** by **preference-based data**:

For each pair of outputs: $y_+ > y_-$

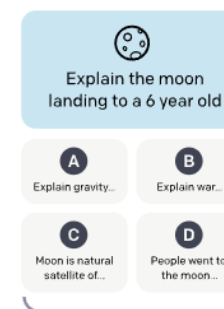
$$L_{RM} = -\log \sigma(r_{\theta}(x, y_+) - r_{\theta}(x, y_-))$$

where $\sigma(\cdot)$ is the Sigmoid function.



Collect comparison data, and train a reward model.

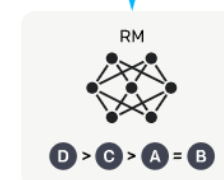
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Summary

- Reward shaping / reward modeling:

$$R^{new}(\cdot) = \alpha R^{env}(\cdot) + \beta \mathbf{R}^{sha}(\cdot)$$

Reward Shaping {

- for **exploration**: assign higher rewards to under-explored states.
- for **exploitation**: assign higher rewards to most important states.
- from **human data**: for open-ended problems.

- Common challenge:

- How to estimate novelty or importance efficiently?
- How to guarantee consistent of the original optimal policy?
- How to balance exploration and exploitation?

RL with an Assistant Reward Agent (ReLara)

• Framework

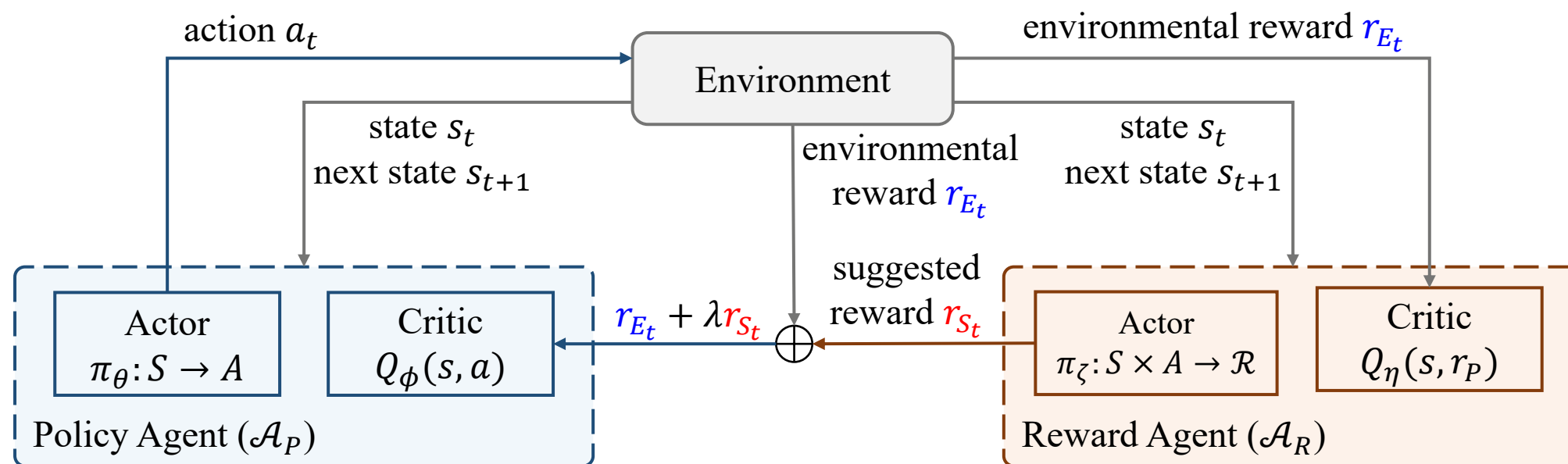
○ Policy agent (PA) $\mathcal{A}_P$:

➤ Given state from the environment, act an action to interact with the environment.

○ Reward agent (RA) $\mathcal{A}_R$: The agent to generate a suggested reward.

➤ Treat the reward generation as a decision-making problem.

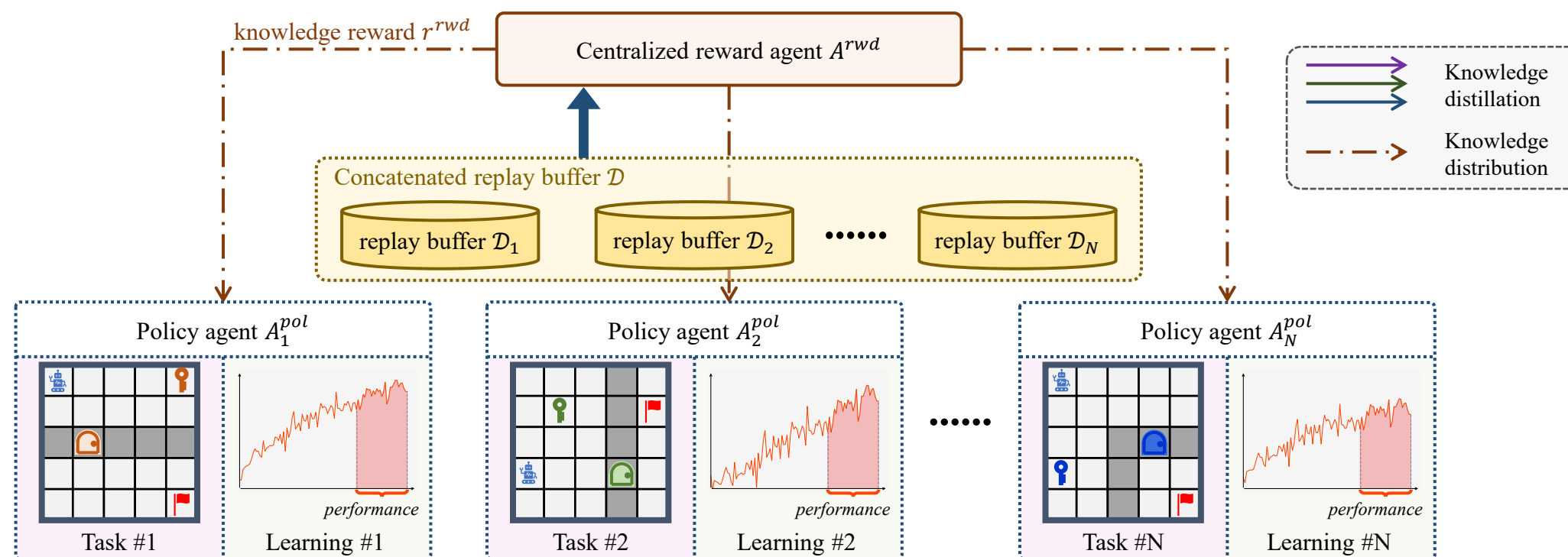
➤ Given the state-action, generate a reward signal.



[1] **Haozhe Ma**, Kuankuan Sima, Thanh Vinh Vo, Di Fu, and Tze-Yun Leong. 2024. Reward Shaping for Reinforcement Learning with An Assistant Reward Agent. In Proceedings of the 41st International Conference on Machine Learning. (ICML 2024)

Multi-Task RL with a Centralized Reward Agent (CenRA)

- **Sharing and transferring the knowledge learned by Reward Agent**
 - The knowledge learned by the Reward Agent can be shared and transferred to similar tasks.
 - A **Centralized Reward Agent (CRA)** to extract and distribute knowledge to multiple tasks.

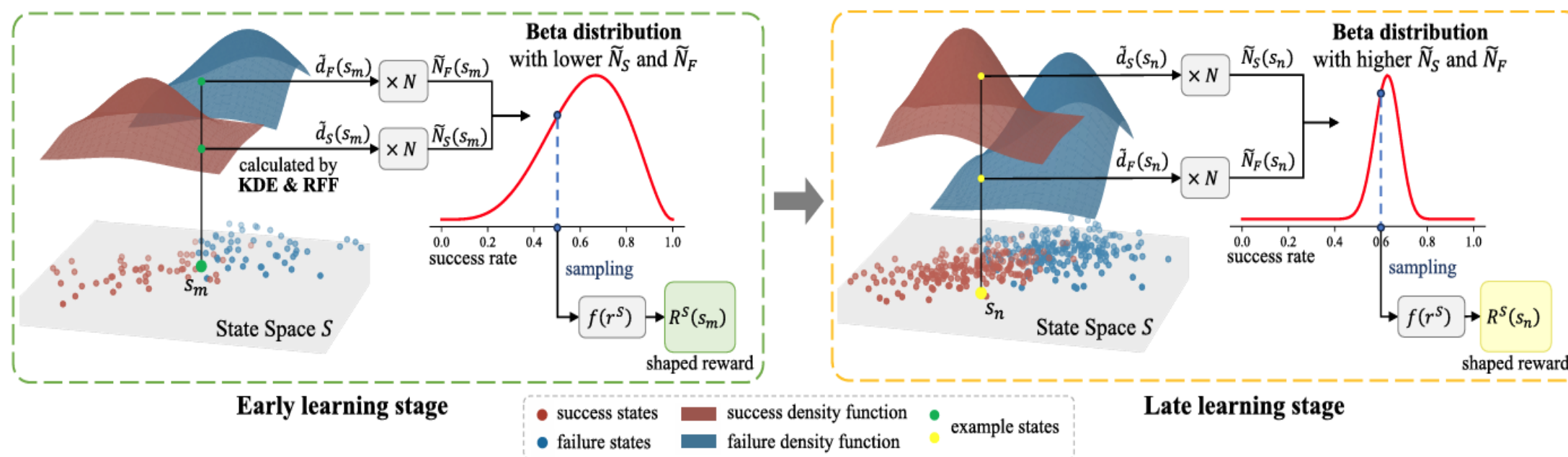


[1] **Haozhe Ma**, Zhengding Luo, Kuankuan Sima, Thanh Vinh Vo, Tze-Yun Leong. Centralized Reward Agent for Knowledge Sharing and Transfer in Multi-Task Reinforcement Learning. In Proceedings of the 39th Annual Conference on Neural Information Processing Systems. (**NeurIPS 2025**)

Self-Adaptive Success Rate based RS (SASR)

• Methodology

- Data accumulates along training processes.
- More data build more confident Beta distributions.



[1] **Haozhe Ma**, Zhengding Luo, Thanh Vinh Vo, Kuankuan Sima, Tze-Yun Leong. 2025. Highly Efficient Self-Adaptive Reward Shaping for Reinforcement Learning. In proceedings of the 13th International Conference on Learning Representations. (ICLR 2025)

Summary

- Reward shaping / reward modeling:

$$R^{new}(\cdot) = \alpha R^{env}(\cdot) + \beta \mathbf{R}^{sha}(\cdot)$$

Reward Shaping

- for **exploration**: assign higher rewards to under-explored states.
- for **exploitation**: assign higher rewards to most important states.
- from **human data**: for open-ended problems.

- Reference

- [1] Bellemare M, Srinivasan S, Ostrovski G, et al. Unifying count-based exploration and intrinsic motivation[J]. Advances in neural information processing systems, 2016, 29.
- [2] Burda Y, Edwards H, Storkey A, et al. Exploration by random network distillation[J]. arXiv preprint arXiv:1810.12894, 2018.
- [3] Wang P, Li L, Shao Z, et al. Math-shepherd: Verify and reinforce llms step-by-step without human annotations[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 9426-9439.
- [4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray et al., “Training language models to follow instructions with human feedback,” Advances in Neural Information Processing Systems, vol. 35, pp. 27 730–27 744, 2022.
- [5] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” Advances in neural information processing systems, vol. 30, 2017.
- [6] Lil’Log: Exploration Strategies in Deep Reinforcement Learning <https://lilianweng.github.io/posts/2020-06-07-exploration-drl/>
- [7] OpenAI Blog: Reinforcement Learning with Prediction-Based Rewards <https://openai.com/index/reinforcement-learning-with-prediction-based-rewards/>

Thank You!